

Éditeur de Formes Géométriques

Rapport de Projet

AMARA Kousay

Avril 2026

Sujet

Micro-éditeur de figures géométriques vectorielles en Java AWT démontrant l'application de design patterns pour une architecture maintenable et extensible.

1. Introduction

J'ai conçu un éditeur vectoriel interactif en Java AWT permettant de créer, modifier, grouper et annuler des opérations sur des formes géométriques. L'objectif était de démontrer l'application de design patterns pour construire une architecture maintenable, extensible et testable.

Le sujet officiel comporte 8 cas d'utilisation. Cette version couvre l'ensemble du périmètre attendu : drag-drop de formes depuis une toolbar, groupage/dégroupage, édition des propriétés, sauvegarde de prototypes via clone, undo/redo, sauvegarde/chargement d'un document et persistance de la toolbar au redémarrage. L'implémentation s'appuie sur 20 tests automatisés, mêlant tests unitaires et test d'intégration.

Contexte : Ce projet a été réalisé en solo. L'implémentation couvre les 8 cas d'utilisation du sujet, avec une sélection multiple réalisée par **Ctrl+clics** successifs ou par rectangle de sélection.

2. Les Six Design Patterns Appliqués

2.1. Composite Pattern

Le problème était simple : comment traiter les formes individuelles et les groupes de formes de la même manière ? J'ai créé une interface **Shape** (Shape.java ligne 4) que tout implémente : Rectangle, RegularPolygon, mais aussi Group.

Group est le nœud composite. Il contient une liste d'autres shapes et implémente les mêmes opérations (move, rotate, etc.). Quand on appelle **move()** sur un groupe, l'opération s'applique récursivement à tous les enfants, même s'ils sont eux-mêmes des groupes imbriqués. Cela a éliminé le besoin de traiter les groupes différemment du reste du code.

C'est particulièrement utile pour les cas d'utilisation 2 et 3 du sujet où on doit grouper et dégroupier des formes. Un groupe peut contenir d'autres groupes sans limite.

2.2. Prototype Pattern

Pour la toolbar, j'avais besoin de cloner des formes avec toutes leurs propriétés. Au lieu de créer des constructeurs complexes, chaque shape implémente une méthode **clone()**.

En regardant le code : Shape.java ligne 19 définit l'interface, et chaque implémentation (Rectangle ligne 52, RegularPolygon ligne 90, Group ligne 120) retourne une copie conforme. La copie

est une vraie duplication des propriétés et, dans le cas de Group, elle clone récursivement tous les enfants.

Cela simplifiait énormément le drag-drop dans la toolbar : on stocke un prototype pour chaque forme, et quand l'utilisateur la traîne, on appelle `clone()` pour créer une instance indépendante. Si l'utilisateur la remet dans la toolbar, on la clone de nouveau.

Le pattern Prototype reste central pour les cas 1 et 5, où la toolbar sert de réservoir de formes réutilisables. En revanche, pour les cas 7 et 8, la persistance repose surtout sur `ShapePersistenceService` et le DTO `ShapeData`, qui sérialisent explicitement les propriétés du modèle.

2.3. Command Pattern

C'était le pattern critique du projet, celui sur lequel le sujet insistait : l'undo/redo. J'ai implémenté Command (`Command.java` ligne 3) comme une interface avec deux responsabilités : `execute()` pour faire l'action et `undo()` pour l'inverser.

Chaque opération dans l'éditeur est une commande concrète : `AddShapeCommand`, `RemoveShapeCommand`, `MoveShapeCommand`, `EditShapeCommand`, `GroupCommand`, `UngroupCommand`, ainsi que `AddPrototypeCommand` et `RemovePrototypeCommand` pour les actions liées à la toolbar. `CommandHistory` (ligne 6) gère le mécanisme avec un double-stack : une pile undo et une pile redo.

Quand l'utilisateur ajoute un premier rectangle, puis un second, puis groupe les deux, le système empile trois commandes. Quand on clique undo 3 fois, chaque commande défait son action dans l'ordre inverse. Redo refait dans le bon ordre. C'est une mécanique classique et fiable.

L'avantage est que chaque commande ne connaît que sa propre logique d'undo. Ajouter une nouvelle commande = ajouter une classe. Aucune modification à `CommandHistory` n'est nécessaire.

Pour le cas 7, après le chargement d'un document, `history.clear()` réinitialise les deux piles pour qu'il soit impossible d'annuler vers l'état précédent l'ouverture du fichier.

2.4. Memento Pattern

Le Memento capture un snapshot de l'état d'une shape : position, taille, rotation, couleur, etc. `ShapeMemento` (ligne 11) stocke ces propriétés.

Ce pattern est utilisé surtout dans `EditShapeCommand` (ligne 12). Avant que l'utilisateur modifie une shape dans le `PropertyDialog`, on crée un memento de l'état original. L'utilisateur édite (la shape change). Après "OK", on crée un second memento avec le nouvel état. Quand on undo, on restaure le memento original.

Dans cette implémentation, le memento porte lui-même la logique de restauration. La méthode `restore()` remet l'état précédent ou suivant sans exposer tous les détails internes à l'historique des commandes.

2.5. Strategy Pattern

J'avais un problème : `PropertyDialog` avait 80+ lignes de code dupliquées pour éditer Rectangle, RegularPolygon, ou Group—le même pattern (créer des champs, récupérer les valeurs, valider) mais avec des champs différents selon le type.

J'ai créé `PropertyEditorStrategy` (ligne 10) comme interface. La méthode `setupFields()` crée les bons champs et `applyChanges()` récupère les valeurs. Puis j'ai écrit trois implémentations : `RectanglePropertyEditor`, `PolygonPropertyEditor`, `GroupPropertyEditor`.

`PropertyDialog` délègue maintenant le travail à la bonne stratégie (choix dynamique). Cela a réduit considérablement le volume de code spécifique dans le dialogue, bien au-delà de la moitié

de l'implémentation initiale. Et si j'ajoute une nouvelle forme à l'avenir, je crée une nouvelle stratégie sans toucher à `PropertyDialog`, avec ajout du cas correspondant dans la factory.

2.6. Observer Pattern

Le modèle et la vue doivent rester découplés. Pour cela, j'ai implémenté un mécanisme d'observation au niveau de la scène.

`SceneListener` définit une interface de notification avec la méthode `onShapeChanged()`. La classe `Scene` maintient une liste de listeners et appelle `notifyListeners()` lors des changements de structure de la scène : ajout ou suppression de formes, réorganisation, et mise à jour de la sélection.

`WhiteboardPanel` implémente `SceneListener`. Quand il reçoit une notification, il déclenche un `repaint()` pour redessiner l'affichage.

Dans l'implémentation actuelle, ce pattern couvre donc bien les changements portés par `Scene`. En revanche, certaines modifications interactives plus fines, comme certains déplacements ou éditions appliqués directement aux formes, provoquent encore un rafraîchissement explicite côté interface. L'Observer est donc présent et utile pour découpler la scène de la vue, mais il ne couvre pas encore automatiquement toutes les mutations internes des objets métier.

Le cas 7 bénéficie aussi de l'Observer : `scene.replaceShapes()` appelle `notifyListeners()` en fin d'exécution, ce qui déclenche automatiquement le redessinage complet de la vue après chargement.

3. Tests et Validation

La suite de tests compte 20 tests essentiels qui couvrent les cas importants.

`CommandTest` contient 8 tests : chaque type de commande critique est testé individuellement, avec un test d'intégration `testCompleteUndoRedoSequence()` qui enchaîne création, groupage et déplacement de formes, validant la mécanique du double-stack, ainsi qu'un test vérifiant la restauration de l'ordre des formes après undo d'un groupage.

`ModelTest` contient 10 tests sur la création, le clonage, le mouvement de formes, et la hiérarchie des groupes imbriqués.

`ShapePersistenceServiceTest` ajoute 2 tests dédiés à la sérialisation logique : sauvegarde/rechargement de formes simples, puis sauvegarde/rechargement d'un groupe imbriqué avec conservation de la hiérarchie et des propriétés essentielles.

La couverture reste cependant centrée sur le noyau modèle/commandes. L'interface AWT, la toolbar, `PropertyDialog`, `SelectionController` et les commandes de prototypes ne sont pas testés automatiquement.

4. Conformité aux Cas d'Utilisation

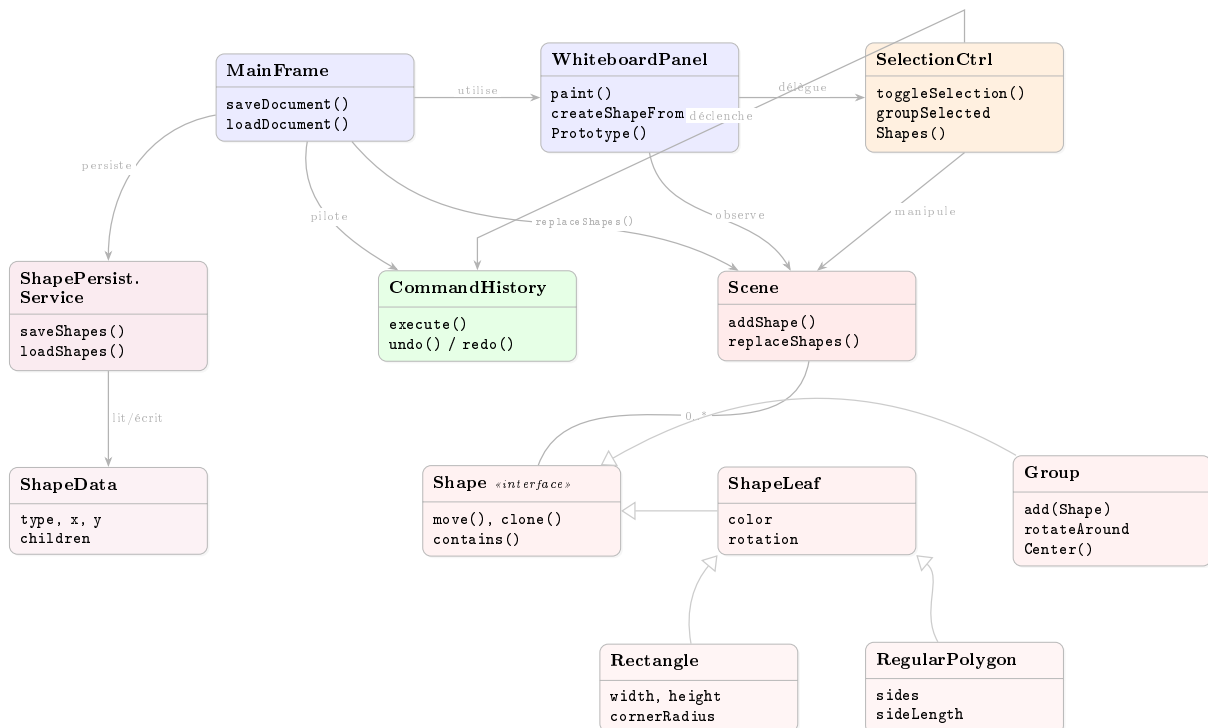
Le sujet comporte 8 cas d'utilisation. Le projet couvre les 8 cas :

1. **Drag-drop toolbar** : `AddShapeCommand` gère l'ajout dans la scène
2. **Groupage** : `Group` et `SelectionController` gèrent la sélection et la création du groupe ; la sélection multiple est réalisée via des `Ctrl+clics` successifs ou par rectangle de sélection
3. **Dégroupage** : `UngroupCommand` décompose le groupe et remet les formes individuelles
4. **Édition propriétés** : `EditShapeCommand` + `PropertyDialog` ; les propriétés éditables incluent position, taille, rotation, couleur, et pour les formes simples le centre de rotation, initialisé par défaut sur le centre de la forme puis modifiable
5. **Drag-drop vers toolbar** : `clone()` + `AddPrototypeCommand` pour ajouter un prototype ; suppression vers la poubelle via `RemoveShapeCommand`

6. **Undo/Redo** : CommandHistory avec double-stack, cœur du système
7. **Sauvegarde/chargement document** : boutons Save/Load dans ControlPanel ; sérialisation des formes de la scène via ShapePersistenceService
8. **Persistance toolbar** : l'état des prototypes est sauvegardé à la fermeture puis restauré au démarrage

5. Diagramme UML

Le diagramme suivant présente une vue UML simplifiée des principales classes et de leurs dépendances.



Cette vue UML met en évidence la hiérarchie des formes, les dépendances entre interface et contrôleur, le rôle de l'historique de commandes, ainsi que la place de la persistance dans le cycle de vie du document et de la toolbar.

Cette organisation maintient une séparation claire des responsabilités et facilite la maintenance.

6. Conclusion

Ce projet a appliqué six design patterns fondamentaux :

- **Composite** pour la hiérarchie de formes
- **Prototype** pour le clonage et la toolbar
- **Command** pour undo/redo
- **Memento** pour sauvegarder/restaurer l'état
- **Strategy** pour l'édition polymorphe des propriétés
- **Observer** pour le découplage modèle-vue

Ces patterns structurent les points les plus importants du projet : hiérarchie de formes, clonage pour la toolbar, annulation/rétablissement des actions, édition selon le type de forme, et découplage partiel entre scène et affichage. L'architecture facilite l'extension, mais certaines

zones restent à adapter lorsqu'on ajoute une nouvelle forme, notamment le renderer, la factory des éditeurs, certains calculs géométriques et la sauvegarde/restauration d'état.

7. Limites Connues

- Le format de persistance est binaire et spécifique au projet ; il n'est pas pensé pour l'interopérabilité.
- Le mécanisme Observer couvre surtout les changements de scène et de sélection ; certaines mises à jour restent déclenchées explicitement par l'interface.
- L'édition des groupes reste plus simple que celle des formes élémentaires : on édite leur centre global, leur rotation et leur échelle, mais pas un pivot distinct.
- L'ajout d'une nouvelle forme reste faisable, mais demande encore des modifications dans plusieurs classes transverses.

Fichiers clés du projet :

- `src/main/java/model/Shape.java` (Composite + Prototype)
- `src/main/java/command/Command.java` et `CommandHistory.java` (Command)
- `src/main/java/command/ShapeMemento.java` (Memento)
- `src/main/java/ui/strategy/PropertyEditorStrategy.java` (Strategy)
- `src/main/java/model/Scene.java` (Observer)
- `src/main/java/persistence/ShapePersistenceService.java` (cas 7 et 8)
- `src/test/java/command/CommandTest.java` (tests undo/redo)